

PULS COFFEE

AI Chat Team Plan

How to Build Puls Coffee with 5 Focused AI Chats

CTO • Backend • iOS • Dashboard • DevOps

You are the CTO.

5 focused Claude Projects.

1 shared specification.

Clear API contracts between every chat.

Sequential execution. No parallel chaos.

Version 1.0 | May 2025

1
CTO

2
Backend

3
iOS

4
Dashboard

5
DevOps

Table of Contents

1. The Concept — Why 5 Chats
2. Team Overview — All 5 Chats
3. The Golden Rules — Every Chat Must Follow
4. Chat 1 — CTO / Architect (this chat)
5. Chat 2 — Backend Engineer
6. Chat 3 — iOS Developer
7. Chat 4 — Staff Dashboard Developer
8. Chat 5 — DevOps Engineer
9. The Contract System — Preventing Chaos
10. Definition of Done — Every Feature
11. Git Workflow — Even Solo
12. Execution Order — Sequential Not Parallel
13. Architecture Freeze Rule
14. Environment Separation
15. What You Already Have — Starting Point

1. THE CONCEPT — WHY 5 CHATS

You are building Puls Coffee with no team — just you, your understanding of code, and 5 focused Claude AI chats acting as specialized engineers. Each chat has one job, one domain, and cannot redesign the architecture.

The Problem with One Chat

- ✗ Context gets too long — chat starts forgetting earlier decisions
- ✗ Mixes backend + iOS + infrastructure concerns — creates confusion
- ✗ Starts contradicting itself after 50+ messages
- ✗ Loses track of which golden rules apply
- ✗ Tries to 'improve' architecture on every response

The Solution — 5 Focused Chats

- ✓ Each chat has exactly one domain — it cannot touch another
- ✓ Each chat has the spec PDF — it knows the full plan
- ✓ Each chat has strict rules — it cannot redesign
- ✓ You are the CTO connecting them — the only human in the loop
- ✓ Contracts flow between chats — preventing shape mismatches
- ✓ Sequential execution — backend finishes before iOS starts

The Real Risk Is Not Code

The danger is not that a chat writes bad code. The danger is that Backend returns one shape, iOS expects another, Dashboard uses different status names, and DevOps deploys wrong env variables. This plan prevents that.

2. TEAM OVERVIEW — ALL 5 CHATS

Chat	Role	Builds	Starts
Chat 1 — CTO	Architect & decision maker	Architecture decisions, API contracts, bug triage, conflict resolution	Now — always active
Chat 2 — Backend	Senior NestJS engineer	NestJS API, PostgreSQL, Stripe, Clover, workers, Telegram	Week 1 — first chat to open
Chat 3 — iOS	Senior SwiftUI developer	All SwiftUI screens, local cache, Stripe SDK, push notifications	Week 4 — after backend done
Chat 4 — Dashboard	Senior React developer	React staff dashboard, live orders, inventory, reports	Week 5 — after backend done
Chat 5 — DevOps	Infrastructure engineer	Docker, AWS ECS, RDS, Redis, SQS, CI/CD, monitoring	Week 3 — after backend works locally

What Every Chat Must Have

- ✓ PulsCoffee_Final_Spec.pdf uploaded to the project
- ✓ The system prompt for its specific role (see sections 4-8)
- ✓ The golden rules list — all 15 rules from the spec
- ✓ The architecture freeze rule — no redesigns without CTO approval
- ✓ Its strict boundary — it cannot touch another chat's domain

Flow Between Chats

```
You (CTO chat)
  ↓ decides what to build next
Backend chat builds endpoint
  ↓ you extract the contract
You paste contract into iOS chat + Dashboard chat
  ↓ they consume the real shape
DevOps chat deploys the working backend
  ↓ all chats test against real staging API
Feature marked DONE
```

3. THE GOLDEN RULES — EVERY CHAT MUST FOLLOW

These rules are pasted into every chat's system prompt. If any chat suggests breaking one, reject it and correct it immediately.

#	Rule	What It Means
1	Stripe webhook = payment truth	iOS NEVER marks an order paid. Only the webhook does.
2	All money = integer cents	650 not 6.50. No floats in business logic. Ever.
3	iOS never calculates prices	Backend calculates. iOS displays what backend returns.
4	Outbox pattern for ORDER_PAID	DB update + event insert in ONE atomic transaction.
5	Three separate status enums	OrderStatus / PaymentStatus / CloverSyncStatus are independent.
6	Clover failure ≠ order failure	POS sync fail ≠ payment fail. Order stays PAID. Alert owner.
7	Cart lives on iOS until checkout	No server cart. POST /checkout sends full cart.
8	Phase 1 = boring and reliable	No AI features. No experiments. No clever logic. Ship it working.
9	Menu loads from cache first	Always. API refresh in background. Never block UI.
10	Checkout button locks on tap	Disabled immediately. Spinner. Reset only on error.
11	JWT in Keychain — never UserDefaults	Security. Non-negotiable on iOS.
12	Feature flags for risky features	AI, pricing, subscriptions default FALSE. Deploy then enable.
13	Every record scoped to location_id	Multi-location from day one. Never hard-code one shop.
14	Staff dashboard before AI	Operations first. AI is Phase 2.
15	No architectural changes without CTO	Raise it in CTO chat. Get approval. Then change.

1

CTO / Architect Chat

This chat — always open — final decisions only

What This Chat Does

- ✓ Makes all architecture and priority decisions
- ✓ Writes and maintains API contracts between chats
- ✓ Resolves conflicts when two chats disagree
- ✓ Triage bugs — decides which chat fixes them
- ✓ Reviews any proposed schema changes before they happen
- ✓ Decides what gets built next and in what order
- ✓ Checks if any chat is contradicting the spec or golden rules
- ✓ Tracks overall progress — what is done, what is next

What This Chat Does NOT Do

- ✗ Write NestJS code — that is Backend chat
- ✗ Write SwiftUI code — that is iOS chat
- ✗ Write React code — that is Dashboard chat
- ✗ Set up AWS — that is DevOps chat
- ✗ Add new features not in the spec without reason
- ✗ Redesign the architecture mid-build

How to Use This Chat

Situation	What to Paste Here
Backend built an endpoint	Paste the response shape → this chat writes the contract
iOS and Backend disagree on a field	Paste both versions → this chat decides which is correct
A production error appears	Paste the error → this chat decides which chat fixes it
You want to add a new feature	Describe it here first → this chat approves or delays it
A chat suggests changing the database	Paste the suggestion → this chat approves or rejects
You're not sure what to build next	Ask here → this chat gives the next clear instruction

System Prompt for This Chat

(This chat already has context — no new prompt needed)

What This Chat Builds

- ✓ NestJS modules: auth, users, locations, menu, pricing, inventory, checkout, orders, payments, refunds, clover, loyalty, notifications, admin, analytics
- ✓ TypeORM entities and all PostgreSQL schema
- ✓ Pricing service — all math in integer cents
- ✓ Checkout service with idempotency guard
- ✓ Stripe PaymentIntent creation and webhook handler
- ✓ Clover sync worker with retry (0s → 30s → 2m → 10m → DLQ)
- ✓ Outbox worker — polls every 1 second
- ✓ Order worker — loyalty points + push + analytics
- ✓ Telegram alert service
- ✓ Feature flags service
- ✓ OpenAPI/Swagger documentation for every endpoint
- ✓ Unit tests for critical services

What This Chat Never Touches

- ✗ SwiftUI or any iOS code
- ✗ React or any dashboard code
- ✗ AWS infrastructure configs
- ✗ Docker or CI/CD setup
- ✗ Any feature not in the spec without CTO approval

System Prompt — Copy This Exactly

You are the senior backend engineer for Puls Coffee.
You build only the NestJS + TypeScript backend.
Follow the uploaded PulsCoffee_Final_Spec.pdf exactly.

Do not suggest GraphQL, microservices, EventBridge,
Apple Watch, AR, or any new product features.

Never break these rules:

1. Stripe webhook is payment truth.
2. All money is integer cents — never floats.
3. iOS never calculates prices — backend calculates everything.
4. ORDER_PAID uses outbox pattern — DB update + event in one transaction.
5. Three separate statuses: OrderStatus, PaymentStatus, CloverSyncStatus.

6. Clover sync failure is NOT order failure – OrderStatus stays PAID.
7. Cart lives on iOS until checkout – no server cart.
8. Phase 1 is boring and reliable – no AI features yet.
9. No architectural changes without CTO chat approval.
10. Every endpoint gets OpenAPI/Swagger documentation.

Your job:

Build clean production-ready NestJS code in small steps.

Always show the exact file path before each code block.

Always provide complete working code – no snippets.

After each step tell me exactly what command to run and what output to expect.

When a module is complete, write the API contract:

endpoint, method, auth, request body, response body, errors.

If anything is unclear, ask before assuming.

Build Order for Backend Chat

Step	Module	What It Produces
1	Entities + migrations	All TypeORM entities, enums, docker-compose — already done ■
2	Auth module	POST /auth/register, /auth/login, /auth/refresh + JWT guards
3	Locations module	Location hours, settings, open/closed validation
4	Menu module	GET /menu with Redis cache, GET /menu/items/:id
5	Pricing module	calculateOrder() in integer cents — all price math here
6	Inventory module	availability tracking, sold-out toggle
7	Checkout module	POST /checkout with idempotency — the heart of the business
8	Payments module	Stripe webhook handler — sets order PAID
9	Workers	outbox.worker, order.worker, clover.worker
10	Admin module	All /admin/* endpoints for staff dashboard
11	Loyalty module	Points earn/redeem, tier upgrades
12	Notifications	Telegram alerts + APNs push stubs
13	Refunds module	Full and partial refund via Stripe
14	Tests	Unit tests for pricing, checkout, state machine

Starting Instruction for Backend Chat

After uploading the spec PDF and existing code zip, say:

"We have the database entities, docker-compose, pricing service, checkout service, stripe service, clover worker, outbox worker, and telegram service already built.

Next step: build the auth module.

Build src/modules/auth/auth.service.ts, auth.controller.ts, and jwt.strategy.ts.

Use AWS Cognito JWT verification.

Separate guards for customers and staff.

Show exact file paths and complete code."

What This Chat Builds

- ✓ Complete SwiftUI project structure (PulsCoffeeApp/)
- ✓ Auth screens: LoginView, RegisterView, AuthViewModel
- ✓ MenuView — loads from local disk cache first, refreshes in background
- ✓ ItemDetailView — modifiers, quantity selector, add to cart
- ✓ CartManager — @StateObject, lives in memory only, no API calls
- ✓ CheckoutView — button locks on first tap, idempotency key generation
- ✓ Stripe payment sheet integration + Apple Pay
- ✓ OrderStatusView — polls GET /orders/:id every 10 seconds
- ✓ OrderHistoryView — paginated, last 10 cached locally
- ✓ LoyaltyView — points balance, tier, transaction history
- ✓ ProfileView — customer settings, push notification toggle
- ✓ MenuCacheService — disk cache, show immediately, refresh silently
- ✓ APIClient.swift — all network calls, single entry point
- ✓ Sentry SDK — initialized in App.init() first line
- ✓ PostHog SDK — behaviour event tracking
- ✓ App Store assets — screenshots, description, privacy policy

What This Chat Never Touches

- ✗ NestJS or any backend code
- ✗ React or any dashboard code
- ✗ AWS or any infrastructure
- ✗ Any price calculation — display only what backend returns
- ✗ Setting payment status — Stripe webhook does that
- ✗ Any AI features — Phase 2 only

System Prompt — Copy This Exactly

You are the senior SwiftUI iOS developer for Puls Coffee.
You build only the SwiftUI iOS app (iOS 16+).
Follow the uploaded PulsCoffee_Final_Spec.pdf exactly.
The backend API is already built – never suggest changing it.

Never break these rules:

1. Menu loads from local disk cache first – always.
API refresh happens silently in background Task{ }.
2. Cart lives in memory only – CartManager is @StateObject.
No API calls when adding or removing items.
POST /checkout sends the full cart on submission.
3. Checkout button locks on first tap:
guard !isProcessing else { return }
isProcessing = true immediately.
Only reset to false on error.
4. iOS never calculates prices.
Display PriceCalculation returned by backend.
No local math anywhere.
5. JWT stored in iOS Keychain (kSecAttrAccessibleWhenUnlocked).
Never UserDefaults. Never ever.
6. Sentry initialized first line of App.init() – before anything else.
7. Order status polled every 10 seconds while active.
Stop polling when status is PICKED_UP, CANCELLED, or FAILED.
8. Idempotency key = SHA256(userId + sorted(cartItemIds) + timestamp).
Sent in every checkout request.
9. Feature flags fetched on launch, cached 5 minutes.
Every risky feature gated behind a flag check.
10. No architectural changes without CTO chat approval.

Your job:

Build clean production SwiftUI code in small steps.
Always show the exact file path before each code block.
Always provide complete working code – no snippets.
After each step tell me what to do in Xcode to test it.
When you need an API endpoint shape, ask me – I will
give you the exact contract. Never assume the shape.

Build Order for iOS Chat

Step	Screen/File	What It Produces
1	Core/	AppState, APIClient, Storage, EventBus
2	Auth/	LoginView, RegisterView, AuthViewModel
3	Menu/	MenuView with cache, MenuCategoryView, MenuViewModel
4	Menu/	ItemDetailView with modifier selection
5	Cart/	CartManager, CartView
6	Checkout/	CheckoutView, CheckoutViewModel, IdempotencyKey
7	Checkout/	Stripe payment sheet + Apple Pay integration
8	Orders/	OrderStatusView with 10s polling
9	Orders/	OrderHistoryView, OrderViewModel

1 0	Loyalty/	LoyaltyView — points, tier, history
1 1	Profile/	ProfileView — settings, push toggle
1 2	Services/	PushNotificationService, AnalyticsService
1 3	App Store	Screenshots x6, description, privacy policy

Starting Instruction for iOS Chat

After uploading the spec PDF, say:

"The NestJS backend is running at `http://localhost:3000`.

Start with the Core/ files:

```
AppState.swift
APIClient.swift
Storage.swift
```

APIClient should handle:

- JWT token from Keychain
- Automatic token refresh
- Error mapping to app error types
- Base URL from config

Show exact file paths and complete code."

Then paste the menu endpoint contract from CTO chat before asking it to build MenuView.

4

Staff Dashboard Chat

React + TypeScript + Tailwind — AWS Amplify

What This Chat Builds

- ✓ React + TypeScript project setup with Tailwind CSS
- ✓ Live Orders screen — real-time queue, 5s auto-refresh, Accept/Ready/Cancel buttons
- ✓ Order Detail screen — full breakdown, items, modifiers, notes, audit trail
- ✓ Inventory Control screen — toggle sold-out instantly, set quantity
- ✓ Shop Controls — pause/resume ordering, set current wait time
- ✓ Refunds screen — full and partial, Manager+ only, reason required
- ✓ Owner Reports — daily/weekly revenue, top items, new vs returning
- ✓ Feature Flags screen — Owner only, toggle without deployment
- ✓ Staff Management — Owner only, add/remove staff, assign roles
- ✓ Auth — separate Cognito pool for staff, role-based UI
- ✓ Role-based visibility — Barista sees less than Manager/Owner
- ✓ AWS Amplify hosting config — auto-deploy from GitHub

What This Chat Never Touches

- ✗ Any SwiftUI or iOS code
- ✗ Any NestJS backend code
- ✗ AWS infrastructure beyond Amplify hosting
- ✗ Stripe directly — all payments go through backend
- ✗ Price calculation — display what backend returns

System Prompt — Copy This Exactly

You are the senior React TypeScript developer for Puls Coffee.
You build only the React staff web dashboard.
Follow the uploaded PulsCoffee_Final_Spec.pdf exactly.
The backend API is already built – never suggest changing it.

Stack: React + TypeScript + Tailwind CSS
Hosted on: AWS Amplify (free tier)
Auth: AWS Cognito (staff user pool, separate from customers)

Build these screens only:

1. Live Orders – 5s auto-refresh, Accept/Progress/Ready/Cancel
2. Order Detail – items, modifiers, notes, full event audit trail
3. Inventory Control – sold-out toggle, quantity, instant effect
4. Shop Controls – pause ordering, set wait time

5. Refunds – full and partial, Manager+ only
6. Reports – daily/weekly, Owner only
7. Feature Flags – Owner only, toggle without deployment

Never break these rules:

1. Never call Stripe directly – all via backend API.
2. Never calculate prices – display what backend returns.
3. Role-based UI – BARISTA sees less than MANAGER and OWNER.
4. Orders auto-refresh every 5 seconds – use polling not websockets for MVP.
5. All API calls include JWT in Authorization: Bearer header.
6. Show loading states on every async action.
7. Show error messages when API calls fail – never fail silently.
8. No architectural changes without CTO chat approval.

Your job:

Build clean production React code in small steps.

Always show the exact file path before each code block.

Always provide complete working code – no snippets.

When you need an API endpoint shape, ask me –

I will give you the exact contract. Never assume the shape.

Build Order for Dashboard Chat

Step	Screen	What It Produces
1	Project setup	React + TypeScript + Tailwind + Amplify config
2	Auth	Cognito login for staff, role detection, protected routes
3	Live Orders	Real-time queue, 5s refresh, status action buttons
4	Order Detail	Full order breakdown, event audit trail
5	Inventory Control	Sold-out toggle, quantity setter
6	Shop Controls	Pause ordering toggle, wait time editor
7	Refunds	Full/partial refund form, Manager+ guard
8	Reports	Daily/weekly charts and tables, Owner only
9	Feature Flags	Toggle list with descriptions, Owner only
10	Deploy	AWS Amplify config, GitHub Actions auto-deploy

Starting Instruction for Dashboard Chat

After uploading the spec PDF, say:

"The backend API is running at <https://staging.pulscoffee.com>.

Start with the project setup and auth:

- React + TypeScript + Tailwind CSS
- AWS Amplify hosting config
- Cognito auth for staff (separate pool from customers)

- Role detection: OWNER / MANAGER / BARISTA
- Protected routes based on role

Show exact file paths and complete code."

Then paste the orders endpoint contract from CTO chat before building the Live Orders screen.

What This Chat Builds

- ✓ Docker + docker-compose for local development — already done ■
- ✓ AWS ECS Fargate task definitions for API + workers
- ✓ AWS RDS PostgreSQL 15 setup with automated backups
- ✓ AWS ElastiCache Redis cluster configuration
- ✓ AWS SQS queues + DLQ configuration
- ✓ AWS S3 bucket + CloudFront distribution for menu images
- ✓ AWS Cognito user pools — customers + staff (separate)
- ✓ AWS Parameter Store entries for all secrets (encrypted)
- ✓ AWS Amplify setup for staff dashboard hosting
- ✓ GitHub Actions CI/CD pipeline — typecheck, test, build, deploy
- ✓ Sentry project setup and DSN configuration
- ✓ PostHog project setup and API key configuration
- ✓ CloudWatch alarms — DLQ messages, RDS CPU, ECS health
- ✓ Three environments: local, staging, production
- ✓ Rolling deployment config — zero downtime

What This Chat Never Touches

- ✗ NestJS application code
- ✗ SwiftUI or iOS code
- ✗ React dashboard code
- ✗ Business logic of any kind
- ✗ Database schema — that is Backend chat

System Prompt — Copy This Exactly

You are the DevOps and infrastructure engineer for Puls Coffee.
You handle AWS, Docker, CI/CD, monitoring, and security only.
Never write application code (NestJS, Swift, React).
Follow the uploaded PulsCoffee_Final_Spec.pdf exactly.

Stack to configure:

- AWS ECS Fargate (NestJS API + worker containers)
- AWS RDS PostgreSQL 15

- AWS ElastiCache Redis
- AWS SQS + DLQ
- AWS S3 + CloudFront
- AWS Cognito (two pools: customers + staff)
- AWS Parameter Store (SecureString – AES-256)
- AWS Amplify (staff React dashboard)
- Docker + docker-compose (local development)
- GitHub Actions (CI/CD pipeline)
- Sentry (error tracking)
- PostHog (analytics)

Never break these rules:

1. Secrets in AWS Parameter Store only.
Never in .env files in production.
Never committed to git. Ever.
2. Rolling deployments – zero downtime always.
New ECS task starts → health check passes → old task stops.
3. RDS automated daily backups always enabled.
Point-in-time recovery enabled.
4. SQS DLQ configured – alert via CloudWatch → SNS → Telegram.
5. All traffic HTTPS only – HTTP rejected at load balancer.
6. Three environments: local, staging, production.
Separate Stripe keys, databases, S3 buckets, push certs.
7. No architectural changes without CTO chat approval.

Your job:

Provide exact infrastructure config files.

Always show the exact file path before each file.

Always explain what each config does and why.

After each step tell me exactly what command to run
and what output to expect.

Build Order for DevOps Chat

Step	Task	What It Produces
1	Local stack	docker-compose for PostgreSQL + Redis — already done ■
2	GitHub Actions	CI pipeline: typecheck → test → build on every PR
3	AWS setup	VPC, subnets, security groups, IAM roles
4	Parameter Store	All secrets entered: Stripe, Clover, Telegram, DB URL
5	RDS	PostgreSQL 15 instance, backups, connection string
6	ElastiCache	Redis cluster, connection string
7	SQS	Order events queue + DLQ, CloudWatch alarm
8	ECS Fargate	Task definitions for API + worker containers
9	ECR	Docker image registry, push workflow

1 0	CloudFront	S3 bucket + CDN for menu images
1 1	Cognito	Customer pool + staff pool, JWT config
1 2	Staging deploy	Full stack deployed to staging environment
1 3	Production	Production deploy after staging QA passes

Starting Instruction for DevOps Chat

After uploading the spec PDF, say:

"The NestJS backend code is complete and tested locally.
We have docker-compose for local development.

Start with the GitHub Actions CI pipeline:

- On every push to any branch: run npm run typecheck
- On every PR: run npm test
- On merge to main: build Docker image, push to ECR, deploy to ECS staging
- Notify via Telegram on deploy success or failure

Show exact file paths (.github/workflows/) and complete YAML. Explain each step."

9. THE CONTRACT SYSTEM — PREVENTING CHAOS

Every time Backend chat builds an endpoint, you extract the contract and paste it into iOS and Dashboard chats before they build anything that consumes it. This is the single most important process in the entire setup.

Contract Template — Use This Every Time

[illegible]

All Contracts to Maintain

Contract File	Owner	Consumed By
---------------	-------	-------------

menu.contract	Backend chat	iOS chat, Dashboard chat
checkout.contract	Backend chat	iOS chat
orders.contract	Backend chat	iOS chat, Dashboard chat
admin-orders.contract	Backend chat	Dashboard chat
inventory.contract	Backend chat	Dashboard chat
loyalty.contract	Backend chat	iOS chat
auth.contract	Backend chat	iOS chat, Dashboard chat
feature-flags.contract	Backend chat	iOS chat, Dashboard chat

Where to Keep Contracts

In your project folder, create:

```
/contracts/
  menu.contract.md
  checkout.contract.md
  orders.contract.md
  admin-orders.contract.md
  inventory.contract.md
  loyalty.contract.md
  auth.contract.md
  feature-flags.contract.md
```

Every time you paste a contract into a chat, paste from these files. Keep them updated. They are the single source of truth.

10. DEFINITION OF DONE — EVERY FEATURE

A feature is not done until every checkbox is ticked. Half-finished features are the biggest cause of chaos in solo projects.

Checkpoint	Who	What to Verify
Backend endpoint exists	Backend chat	Route responds correctly with test curl/Postman
Input validation works	Backend chat	Invalid requests return correct 400 errors
Error handling done	Backend chat	Sentry captures errors, no unhandled exceptions
Contract written	CTO chat	Contract file updated in /contracts/ folder
Unit tests pass	Backend chat	npm test passes with no failures
OpenAPI updated	Backend chat	Swagger docs show the new endpoint
iOS integrated	iOS chat	Feature works in iOS simulator against real API
Dashboard integrated	Dashboard chat	Feature works in browser if applicable
Deployed to staging	DevOps chat	Feature live on staging environment
Manual QA passed	You	Tested by hand on real device or browser
No console errors	You	Zero red errors in Xcode / browser console
Committed and merged	You	Feature branch merged to main via PR

11. GIT WORKFLOW — EVEN SOLO

Never commit directly to main. Even working alone. Feature branches give you clean history, easy rollback, and visible progress.

Branch Naming Convention

```
feature/auth-module
feature/menu-module
feature/checkout-service
feature/stripe-webhook
feature/clover-sync
feature/outbox-worker
feature/ios-menu-screen
feature/ios-checkout
feature/dashboard-live-orders
feature/devops-ci-pipeline
feature/devops-staging-deploy

fix/stripe-webhook-signature
fix/clover-retry-logic
fix/menu-cache-ttl
```

Workflow Per Feature

1. `git checkout main && git pull`
2. `git checkout -b feature/menu-module`
3. Build the feature with Backend chat
4. `git add . && git commit -m "feat: add menu module with Redis cache"`
5. Test locally
6. `git push origin feature/menu-module`
7. Open PR on GitHub → merge to main
8. Delete the branch
9. Update contract files if API changed
10. Notify iOS/Dashboard chats if they need to consume it

Commit Message Format

```
feat: add checkout service with idempotency guard
fix: correct tax calculation rounding in pricing service
feat: add Clover sync retry with dead letter queue
fix: menu cache TTL not resetting on item update
feat: add iOS CartManager with in-memory state
feat: add dashboard live orders with 5s auto-refresh
chore: update docker-compose postgres version
docs: update checkout contract with error codes
```

12. EXECUTION ORDER — SEQUENTIAL NOT PARALLEL

Do NOT open iOS, Dashboard, and DevOps chats all at once. Backend must be done and working before iOS starts. Sequential integration prevents the biggest risk: half-built parallel features.

Weeks 1-2	Backend Only	<i>Chat 2 active only</i>	Build all NestJS modules. Every endpoint working and tested locally with curl/Postman. All contracts written. Docker-compose running. TypeScript compiles clean. Tests pass.
Week 3	DevOps	<i>Chat 5 active</i>	GitHub Actions CI pipeline. AWS staging environment. Deploy backend to ECS staging. Test Stripe webhooks on staging. Test Clover sync. All secrets in Parameter Store.
Week 4	iOS Starts	<i>Chat 3 active</i>	iOS connects to REAL staging API from day one. No mock data. Menu screen → Cart → Checkout → Order status. Each screen tested against live API.
Week 5	Dashboard Starts	<i>Chat 4 active</i>	Dashboard connects to same staging API. Live orders → Inventory → Controls → Refunds. Test with real orders from iOS.
Weeks 6-7	Integration QA	<i>All chats active</i>	Full end-to-end: iOS order → Clover sync → READY push → staff dashboard → pickup. Bug fixes. Rate limiting tests. Load test checkout. Fix anything broken.
Week 8	App Store	<i>All chats available</i>	App Store assets. TestFlight beta with real Puls Coffee customers. Fix App Store review feedback. Production deploy. Go live.

The Golden Execution Rule

```
Backend feature FINISHED
  ↓
Contract WRITTEN and saved to /contracts/
  ↓
iOS integrates the feature
  ↓
Dashboard integrates the feature
  ↓
Deployed to staging
  ↓
Manual QA passed
  ↓
NEXT feature begins

Sequential. Not parallel. Not skipping steps.
```


13. ARCHITECTURE FREEZE RULE

The architecture is decided. It is frozen. The biggest risk in AI-assisted development is that each chat tries to 'improve' the design. This rule prevents that.

Allowed Changes — CTO Chat Approves

- ✓ Production bug that requires a structural fix
- ✓ Security vulnerability that requires architectural change
- ✓ Scalability bottleneck proven by real load data
- ✓ Apple App Store rejection that requires code restructure

NOT Allowed — Reject Immediately

- ✗ Any chat suggesting we switch to GraphQL or AppSync
- ✗ Any chat suggesting microservices or splitting the API
- ✗ Any chat suggesting EventBridge for more event types
- ✗ Any chat suggesting DynamoDB instead of PostgreSQL
- ✗ Any chat suggesting new tables not in the spec
- ✗ Any chat suggesting Phase 2 AI features in Phase 1
- ✗ Any chat suggesting a new tech stack tool

If Any Chat Suggests an Architecture Change

1. Stop the chat immediately
2. Do not implement the suggestion
3. Come to CTO chat with:
 - What the chat suggested
 - Why it suggested it
 - The problem it was trying to solve
4. CTO chat decides:
 - Approve: then communicate to the right chat
 - Reject: give the chat a simpler solution
 - Delay: add to Phase 2 backlog
5. Only then continue

14. ENVIRONMENT SEPARATION

Three environments. Separate everything. Mixing staging and production will eventually send test push notifications to real customers or charge real cards with test orders.

Resource	Local	Staging	Production
Database	docker-compose PostgreSQL	AWS RDS (small)	AWS RDS (production)
Redis	docker-compose Redis	AWS ElastiCache	AWS ElastiCache
Stripe keys	sk_test_xxx	sk_test_xxx	sk_live_xxx
Stripe webhooks	localhost (Stripe CLI)	staging.pulscoffee.com	api.pulscoffee.com
Clover	Test merchant	Test merchant	Real merchant
S3 bucket	N/A	puls-staging-images	puls-prod-images
APNs certs	Simulator (no push)	Development cert	Production cert
Telegram alerts	Your test chat	Your test chat	Owner's real chat
Sentry	Optional	Staging project	Production project
Domain	localhost:3000	staging.pulscoffee.com	api.pulscoffee.com

Parameter Store Structure

```
/pulscoffee/local/stripe/secret_key    ← sk_test_xxx
/pulscoffee/local/clover/api_key       ← test key
/pulscoffee/staging/stripe/secret_key   ← sk_test_xxx
/pulscoffee/staging/clover/api_key      ← test merchant
/pulscoffee/staging/telegram/token      ← your test bot
/pulscoffee/prod/stripe/secret_key      ← sk_live_xxx
/pulscoffee/prod/clover/api_key         ← real merchant key
/pulscoffee/prod/telegram/token         ← owner's real bot
/pulscoffee/prod/db/url                 ← production RDS URL
```

All stored as SecureString. AES-256 encrypted. Never in .env in production.

15. WHAT YOU ALREADY HAVE — STARTING POINT

This is what was built in the CTO chat. Give the Backend chat the zip file as a starting point.

Already Built — Backend

- ✓ `src/database/entities.ts` — ALL TypeORM entities and enums (every table from spec)
- ✓ `src/database/data-source.ts` — TypeORM DataSource config
- ✓ `src/database/seeds/feature-flags.seed.ts` — feature flag seed data
- ✓ `src/modules/pricing/pricing.service.ts` — all price math in integer cents
- ✓ `src/modules/checkout/checkout.service.ts` — full checkout flow with idempotency
- ✓ `src/modules/payments/stripe.service.ts` — PaymentIntent + webhook handler
- ✓ `src/workers/outbox.worker.ts` — polls every 1 second, guaranteed delivery
- ✓ `src/workers/clover.worker.ts` — retry (0s→30s→2m→10m) + DLQ + Telegram
- ✓ `src/workers/notification.worker.ts` — push notification dispatcher
- ✓ `src/modules/notifications/telegram.service.ts` — owner alerts
- ✓ `src/app.module.ts` — full NestJS module wiring
- ✓ `src/main.ts` — entry point with Sentry init first
- ✓ `src/instrument.ts` — Sentry instrumentation
- ✓ `docker-compose.yml` — PostgreSQL 15 + Redis 7
- ✓ `.env.example` — all required environment variables
- ✓ `tsconfig.json` — TypeScript config
- ✓ `package.json` — all dependencies
- ✓ `README.md` — setup instructions

Still To Build — Backend Chat

- ✗ Auth module (JWT strategy, Cognito integration, role guards)
- ✗ Locations module (hours service, open/closed validation)
- ✗ Menu module (controller, service, Redis cache layer)
- ✗ Inventory module (sold-out toggle, quantity tracking)
- ✗ Orders module (state machine, order history)
- ✗ Admin module (all staff endpoints)
- ✗ Loyalty module (points, tier upgrades, transaction log)
- ✗ Refunds module (full and partial via Stripe)

- ✗ Clover menu import service
- ✗ Unit tests for pricing, checkout, state machine
- ✗ OpenAPI/Swagger documentation

Next Immediate Action

1. Create a new Claude Project called "Puls Coffee – Backend"
2. Upload PulsCoffee_Final_Spec.pdf to the project
3. Upload puls-coffee-backend.zip to the project
4. Paste the Backend system prompt from Section 5 of this document

5. Send this first message:

"We have the database entities, docker-compose, pricing service, checkout service, stripe service, clover worker, outbox worker, and telegram service already built. TypeScript compiles clean.

Next: build the auth module.

Files needed:

```
src/modules/auth/auth.controller.ts
src/modules/auth/auth.service.ts
src/modules/auth/jwt.strategy.ts
src/modules/auth/roles.guard.ts
```

Use AWS Cognito JWT verification.

Separate customer auth and staff auth.

Staff roles: OWNER, MANAGER, BARISTA.

Show exact file paths and complete code."

6. Backend chat builds. You test. You commit.
7. Repeat for each module in the build order.
8. Backend done → write contracts → open DevOps chat.
9. DevOps staging → open iOS chat.
10. iOS done → open Dashboard chat.

Five chats. One spec. Clear contracts. Sequential execution.

You are the CTO. The chats are your engineering team.

Build it.

Puls Coffee — AI Chat Team Plan | Version 1.0 | May 2025